

AI 工作流中的 Workflow、Graph 与 Loop：从概念到实现

👤 Guide 📖 AI 应用开发 📄 约 6778 字 ⌚ 大约 23 分钟

很多刚上手 AI 工作流的开发者都有过类似的困惑：这不就是传统工作流换了个壳吗？为什么不用 Camunda、Temporal 这些成熟引擎？甚至觉得把几个 Prompt 用 if-else 串起来就算“工作流”了。

但真正上手做项目后，这些想法很快会被现实打脸。LLM 的输出天然不确定，单次生成往往不达标，工具调用随时可能失败，上下文窗口还有硬上限。你需要的不是“跑一遍就完事”的线性流程，而是一套能**动态决策、自动修正、可控收敛**的执行机制。

今天这篇文章就来梳理 AI 工作流中三个核心概念——**Workflow、Graph、Loop**，帮你建立从概念到实现的完整认知。本文约 1w 字，建议收藏，通过本文你将搞懂：

1. **为什么 AI 系统需要工作流**：单轮对话和固定流程为什么不够用？动态决策、自动修正、可控收敛分别解决什么问题？
2. **★ Workflow、Graph、Loop 三者的层次关系**：Workflow 是目标与过程，Graph 是结构与载体，Loop 是图上的控制模式——三者如何协作？
3. **★ Graph 的核心元素**：Node（节点）、Edge（边）、State（状态）分别是什么？条件边、动态路由、循环边有何区别？State 的更新策略怎么选？
4. **★ Loop 的设计要点**：固定次数循环 vs 条件驱动循环、嵌套循环的独立性、安全边界的三要素。
5. **★ 从概念到代码**：Spring AI Alibaba 和 LangGraph 的概念映射表 + 完整的“生成→审核→修改”工作流代码实现。
6. **工作流设计的分水岭**：高抽象 vs 低抽象，Node、Edge、State 的抽象原则。

📌 **系列阅读**：本文是 AI Agent 系列的一部分，相关文章：

- [AI Agent 核心概念：Agent Loop、Context Engineering、Tools 注册](#)
- [大模型提示词工程实践指南](#)
- [上下文工程实战指南：让 Agent 少犯蠢的工程方法论](#)
- [万字详解 Agent Skills：是什么？怎么用？和 Prompt、MCP 有什么区别？](#)
- [万字拆解 MCP，附带工程实践](#)
- [一文搞懂 Harness Engineering：六层架构、上下文管理与一线团队实战](#)

一、为什么 AI 系统会需要 workflow

单轮对话虽然可以回答问题，但很难稳定地交付结果。在真实场景中，一个完整任务往往不仅仅是“生成答案”，还包含检索信息、调用工具、输出结构化结果、质量检查、失败重试，以及在结果不满意时进行多轮修正。这些行为本身就是系统结构的一部分，靠一段超长 Prompt 解决不了，需要一种**可分支、可循环、可观测**的执行路径。

传统软件流程通常是确定性的：输入固定、步骤固定、输出相对稳定。但 LLM 的特点恰恰相反——它“能力很强，但不完全稳定”。它可能答非所问、格式错误、产生幻觉，或者在调用工具时失败。这就引出了三个核心问题：

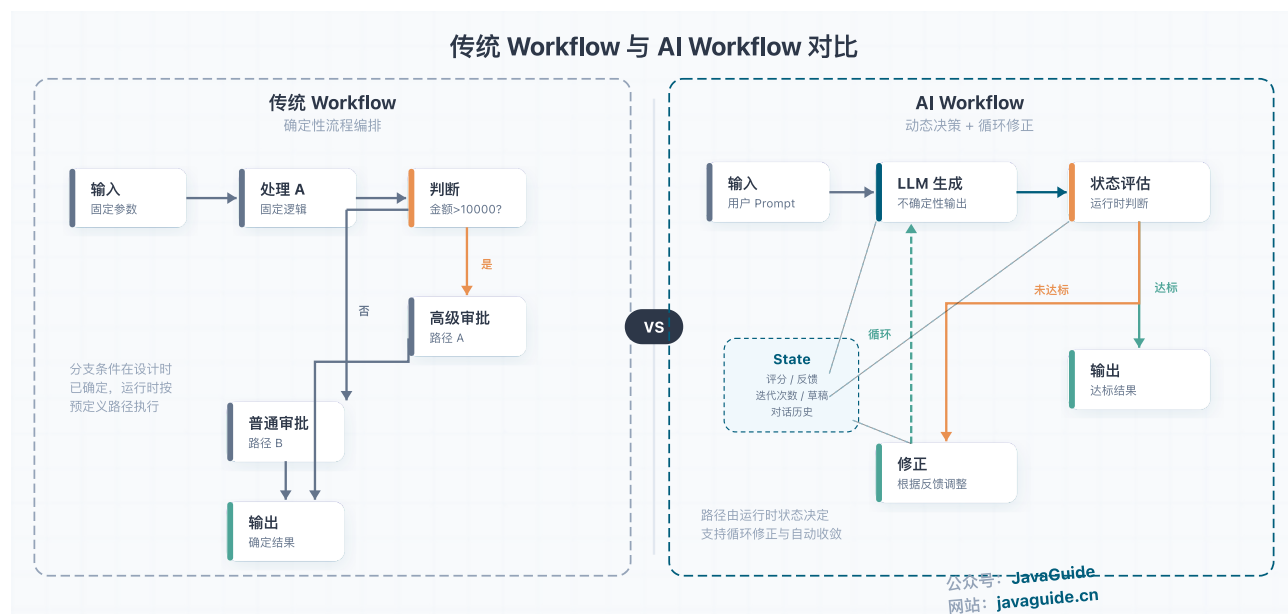
1. 下一步并不唯一，需要根据当前结果动态决策路径；
2. 当结果不理想时，系统需要自动修正，而不是直接失败；
3. 中间状态必须被记录，否则难以调试、追踪与恢复。

这也是为什么 AI 系统需要 workflow 思维。

以一个简单例子来看：当我们让 AI 写一篇文章时，一次生成的结果往往不够理想。直觉做法是手动复制结果，再附加新要求继续提问，但这种方式既不高效，也会快速消耗上下文。如果将这一过程结构化为“**审查** → **修改** → **再审查**”的循环，并设定停止条件（如达到质量标准或触达迭代上限），就能显著提升稳定性。

说到底，workflow 就是把一次性的生成过程，变成一个**可迭代、可收敛、可控制**的系统化流程。

二、工作流是什么：从传统 Workflow 到 AI Workflow



上图可以直观看到两类工作流的差异：传统 Workflow 更偏向“固定步骤 + 明确分支”的过程编排；AI Workflow 则更依赖运行时的状态（State）来动态决定下一步，并通过循环（Loop）把“生成—评估—修正”变成可收敛的过程。

2.1 传统工作流：在做什么？

先说基本定义：**Workflow** 就是为了完成某个目标，把任务拆成若干步骤，并规定这些步骤如何协作推进。它回答的问题是：“这件事怎么做完？”

在传统工作流体系中，流程设计通常强调**确定性与可预测性**。以 BPMN 2.0 规范为代表的主流工作流引擎（如 Camunda、Temporal、Apache Airflow）早已支持并行网关、包容网关、子流程、补偿事务等非线性控制结构，远非简单的线性顺序。但这些控制逻辑通常在设计时就已经确定，运行时按照预定义路径执行。

AI 工作流与传统工作流的关键差异在于：路径选择依赖于运行时生成内容的质量评估，且同一节点可能因输出不确定性而需要反复执行。例如审批流程、订单流转、ETL 数据管道等传统场景中，分支条件是明确的（金额 > 10000 走高级审批）；而 AI 场景中，“生成结果是否达标”这个判断本身就需要运行时评估，且评估结论可能驱使流程回到之前的步骤反复修正。

2.2 AI 工作流：为什么一定会走向 Graph、Loop

到了 AI 场景，同样的“流程”一词，含义不太一样了。相比传统工作流强调的顺序性与确定性，AI 工作流需要处理的是一个充满不确定性的执行环境。我们面对的不再只是“按步骤执行”，还包括：

- 结果是否达标要在**运行时**判断。
- 是否需要继续重试，要由**当前状态**决定。
- 某一步失败后，系统不再是简单的报错然后结束，而是考虑是否应该降级、回退或换一种策略。
- 节点之间传递的不只是参数，还包括上下文、草稿、评分、错误信息、历史轮次等**状态**。

所以 AI Workflow 与传统 Workflow 的差异，不在于“有没有流程”，而在于它更强调动态决策和状态驱动。一旦我们想要表达“下一步不唯一”或者“不满意就再来一轮”，线性列表就不够用，自然会落到 Graph（结构）与 Loop（回溯）这两类概念上。

三、Graph（图）是工作流的结构表达（重要）

沿用贯穿案例：假如我们要搭一条「生成初稿 → 质量审核 → 不达标则修改 → 再回到审核」的路径。这里每一步对应图的 **Node**，步骤之间的走向由 **Edge** 表达，整条链路读写的共享上下文就是 **State**。

图里最基础的元素有三个：

- **Node（节点）**：表示一个执行单元，其主要有三大功能：读取状态（State）、执行业务逻辑并加工状态、将加工好的状态放回。在文章审核例子里，典型有「生成初稿」「质量审核」「按反馈修改」；此外还可以扩展检索、格式校验、人工审批等。
- **Edge（边）**：是流程图中的控制流抽象，用于描述节点之间的执行路径及其触发条件，决定流程在运行时如何在不同节点之间进行调度与跳转。常见的边类型如下：

边的类型	解释
顺序边 (Sequential Edge)	节点按固定顺序执行，执行完当前节点后直接进入下一个节点，不依赖条件或状态判断。

边的类型	解释
条件边 (Conditional Edge)	在设计时定义的有限候选路径中，根据运行时状态（State）选择其一。候选目标节点在设计时确定，运行时只做选择。Spring AI Alibaba 通过 <code>addConditionalEdges()</code> 并传入候选节点映射实现。
动态路由 (Dynamic Routing)	目标节点不在设计时完全预定义，而是由运行时逻辑（如 LLM 决策、map-reduce 分发）动态确定，候选集合可以是开放的。例如 LangGraph 的 <code>Send</code> API 可以在运行时动态决定向某个节点发起多少次并行调用。
循环边 (Loop Edge)	节点可以回到自身或前序节点重复执行，用于重试、迭代优化或循环推理，直到满足终止条件，通常是由条件边与顺序边结合形成。
终止边 (Terminal Edge)	将流程引导至结束状态，不再继续执行后续节点，用于输出最终结果或结束工作流。
并行边 (Parallel Edge)	一个节点同时分发到多个后续节点并行执行，用于多任务处理、RAG/工具并发等场景。

- **State (状态)**：表示在流程执行过程中持续被读写的共享上下文，是节点之间真正传递的“工作记忆”。它本质上是一个**键值对数据结构**（类似 Java 的 `Map<String, Object>`、Python 的 `dict`、TypeScript 的 `Record<string, any>`），用于在各节点之间传递和修改数据。

需要注意的是，State 的设计不仅涉及“存什么”，还涉及“怎么更新”。在实际的工作流框架中，不同字段通常有不同的更新语义：

- **覆盖 (Replace)**：新值直接替换旧值。适用于单值字段，如分类结果、当前状态。在 Spring AI Alibaba 中对应 `ReplaceStrategy`，在 LangGraph 中对应无 reducer 的默认行为。
- **追加 (Append)**：新值追加到已有列表。适用于累积型字段，如对话历史 (messages)。在 Spring AI Alibaba 中对应 `AppendStrategy`，在 LangGraph 中对应 `Annotated[list, operator.add]`。
- **自定义合并 (Custom Reducer)**：通过自定义函数决定合并逻辑，例如 LangGraph 的 `add_messages` 会根据消息 ID 进行追加或更新。

当多个并行节点同时写入同一个使用覆盖语义的字段时，会出现竞态问题（LangGraph 会抛出 `INVALID_CONCURRENT_GRAPH_UPDATE` 错误）。因此，设计 State 时需要提前规划哪些字段可能被并行写入，并为它们选择合适的更新策略。

下面是一些常用的状态字段（可根据实际业务自由扩展，不必拘泥于样例）：

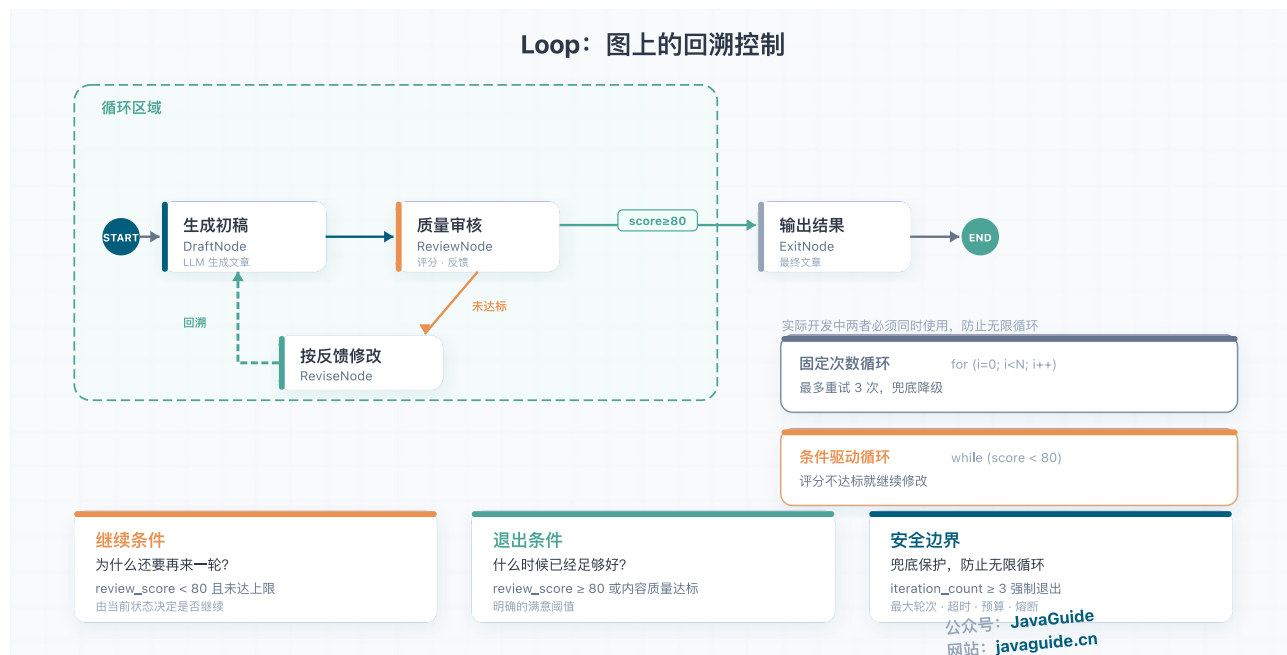
Key（字段名）	Value 类型	说明	生命周期
input	String	用户输入问题	全流程
messages	List	对话历史	全流程
retrieval_result	List	RAG 检索结果	中间
tool_result	Object	工具调用结果	中间
llm_response	String	LLM 原始输出	中间
intermediate_steps	List	中间执行步骤记录	全流程
next_step	String	控制流跳转节点（可选，部分框架如 Spring AI Alibaba 通过此字段配合条件边实现路由；其他框架如 LangGraph 通过条件边函数返回值路由，无需此字段）	当前执行
output	String	最终输出结果	结束

如果只看 Node 和 Edge，我们会得到一张“能跑起来的路径图”；而把 State 一起放进来，我们才真正拥有了一张“可以在运行时做决策的图”。

总之图结构比线性结构更贴近 AI 系统的真实形态，因为很多 AI 应用的控制流本来就是图，只是早期常被临时写成 if-else 、重试逻辑或分散在不同模块里的状态机。

四、Loop 是 Graph 上的回溯能力（重要）

在同一套「文章审核」里：审核不通过时，控制流不应结束，而应沿某条边回到「修改」或「重新生成」——这就是 Loop 在业务上的含义。技术上，它表现为图上的回边（Back Edge）。



很多人第一次接触 AI 工作流时，会把 Loop 理解成“多跑几次”。这不算错，但还不够准确。更准确地说：**Loop 是图结构上的一种控制模式**。当某条边根据当前状态把控制流送回先前节点时，就形成了 Loop，正如上图所示，重点在判断是否达标，在循环的内部 LLM 会根据提示词的要求对结果进行“评分”，如果满足就会输出，否则“打回重写”。

常见的 Loop 主要有两种：

1. **固定次数循环**：更像 for 。例如“最多重试 3 次”。
2. **条件驱动循环**：更像 while 。例如“只要评分低于 80 分，就继续修改”。

AI 场景里，第二类通常更有代表性。因为“跑几次”往往不是先验确定的，而是由内容质量、工具执行结果、外部反馈共同决定的。但是实际开发中两者必须同时使用，因为 LLM 的不确定性可能会导致生成的内容一直不合格，此时我们就需要参考固定次数循环思想对内容进行降级兜底处理。

在实际工程中，还经常遇到**嵌套循环**的情况：外层循环负责“质量迭代”（生成 → 审核 → 修改），内层循环负责“工具重试”（某个节点内部调用外部 API 失败后的指数退避重试）。这两层循环的作用域、终止条件和计数器是独立的——内层重试耗尽不应影响外层的迭代预算，

外层退出也不意味着内层可以无限制重试。设计嵌套循环时，需要为每层明确独立的退出条件和安全边界。

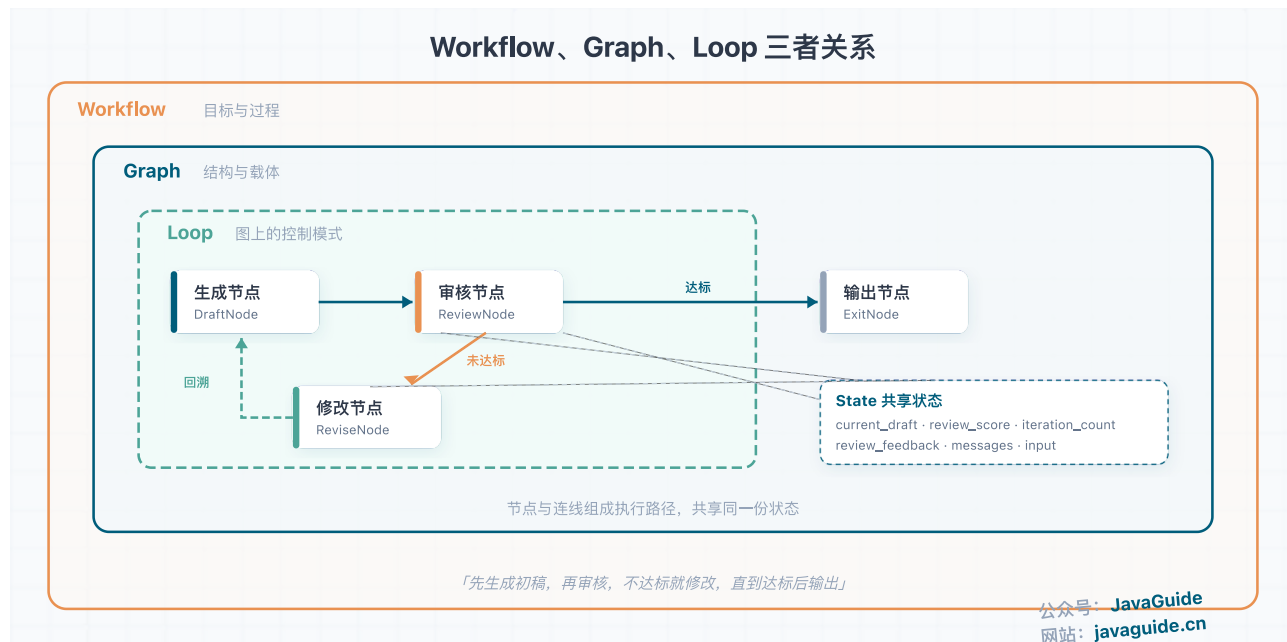
总之，一个可靠的 Loop 一定包含三件事：

- 继续条件：为什么还要再来一轮。
- 退出条件：什么时候已经足够好，可以结束。
- 安全边界：最大轮次、超时、预算、熔断条件。

如果没有这些约束，Loop 很容易从“自我修正”变成“无限打转”。

仍然放回文章审核的例子，Loop 不只是“多试几次”，它是“审核结论驱动下一跳”。只有当评分未达标、且还没超过最大轮次时，流程才会从 ReviewNode 回到 ReviseNode；一旦达到阈值或触发边界条件，就应该退出并给出结果。这时我们看到的就不只是循环，而是一种可控的回溯机制。

五、概念整合：把 Workflow、Graph、Loop 串起来



可以用一句话收束三者的层次关系：**Workflow** 是目标与过程，**Graph** 是结构与载体，**Loop** 是图上的控制模式。

继续沿用同一个“写文章并审核”的例子：

- 当我们说“先生成初稿，再审核，不达标就修改，直到达标后输出”，我们描述的是 **Workflow**。

- 当我们把 生成节点 → 检查节点 → 修正节点 → 检查节点 画成节点与连线，并让它们共享同一份状态时，我们得到的是 **Graph**。
- 当我们规定“审核不通过就回到修改，直到评分达标或达到上限”为止，我们定义的就是 **Loop**。

这三者是同一件事的三个观察角度：Workflow 关注任务目标，Graph 关注结构组织，Loop 关注回溯控制。

六、从概念到实现：框架映射与代码示例

前面建立了 Node、Edge、State 的概念模型，接下来看这些概念如何映射到具体的框架。以下以 Spring AI Alibaba Graph（Java 生态）和 LangGraph（Python 生态）为例。

概念映射表

概念	Spring AI Alibaba	LangGraph
状态 (State)	OverAllState + KeyStrategyFactory	TypedDict + Annotated[reducer]
State 覆盖语义	ReplaceStrategy	默认（无 reducer）
State 追加语义	AppendStrategy	Annotated[list, operator.add]
节点 (Node)	NodeAction 接口	函数 / Runnable
顺序边	addEdge(source, target)	add_edge(source, target)
条件边	addConditionalEdges(source, fn, map)	add_conditional_edges(s, fn)
循环	条件边回指先前节点 / LoopAgent	条件边回指先前节点
固定次数循环	LoopMode.count(N)	自行维护计数器
条件驱动循环	LoopMode.condition(predicate)	条件边 + while 逻辑

概念	Spring AI Alibaba	LangGraph
持久化	MemorySaver / RedisSaver 等	MemorySaver / SqliteSaver
人机协同	interruptBefore() + updateState()	interrupt_before + update_state
编译执行	StateGraph.compile(CompileConfig)	StateGraph.compile()

实现示例：用 Spring AI Alibaba 构建文章审核 workflow

以下代码展示如何用 Spring AI Alibaba Graph 实现贯穿全文的“生成 → 审核 → 修改”工作流。

第一步：定义状态和更新策略

```
1 // 配置状态键策略：控制每个字段如何更新                                     java
2 public static KeyStrategyFactory createKeyStrategyFactory() {
3     return () -> {
4         HashMap<String, KeyStrategy> strategies = new HashMap<>();
5         strategies.put("input", new ReplaceStrategy()); //
6         用户输入
7         strategies.put("messages", new AppendStrategy()); //
8         对话历史 (追加)
9         strategies.put("current_draft", new ReplaceStrategy()); //
10        当前草稿 (覆盖)
11        strategies.put("review_score", new ReplaceStrategy()); //
12        审核评分 (覆盖)
13        strategies.put("review_feedback", new ReplaceStrategy());
14        // 审核反馈
15        strategies.put("iteration_count", new ReplaceStrategy());
16        // 迭代计数
17        strategies.put("output", new ReplaceStrategy()); //
18        最终输出
19        strategies.put("next_node", new ReplaceStrategy()); //
20        路由控制
21        return strategies;
22    };
23 }
```

注意 messages 使用 AppendStrategy（对话历史持续追加），而 current_draft 使用 ReplaceStrategy（每次修改覆盖旧版本）。

第二步：实现节点

```
1 // 生成初稿节点 java
2 public static class DraftNode implements NodeAction {
3     private final ChatClient chatClient;
4
5     public DraftNode(ChatClient.Builder builder) {
6         this.chatClient = builder.build();
7     }
8
9     @Override
10    public Map<String, Object> apply(OverAllState state) throws
11    Exception {
12        String input = state.value("input").map(v -> (String)
13    v).orElse("");
14        String feedback = state.value("review_feedback").map(v ->
15    (String) v).orElse(null);
16
17        String prompt = feedback != null
18            ? String.format("根据以下反馈修改文章：%s\n\n反馈意见：%s",
19    input, feedback)
20            : String.format("请根据以下要求撰写文章：%s", input);
21
22        String draft =
23    chatClient.prompt().user(prompt).call().content();
24
25        return Map.of(
26            "current_draft", draft,
27            "next_node", "review"
28        );
29    }
30 }
31
32 // 质量审核节点
33 public static class ReviewNode implements NodeAction {
34     private final ChatClient chatClient;
35
36     public ReviewNode(ChatClient.Builder builder) {
37         this.chatClient = builder.build();
38     }
```

```
39
40     @Override
41     public Map<String, Object> apply(OverAllState state) throws
42     Exception {
43         String draft = state.value("current_draft").map(v ->
44         (String) v).orElse("");
45         int count = state.value("iteration_count").map(v -> (int)
46         v).orElse(0);
47
48         String prompt = String.format(
49             "请评估以下文章质量，给出 0-100 的评分和改进建议。\\n" +
50             "以JSON格式返回：{\\\"score\\\": 85, \\\"feedback\\\":
51             \\\"...\\\"}\\n\\n%s", draft);
52
53         String response =
54         chatClient.prompt().user(prompt).call().content();
55         // 解析评分和反馈（实际项目中使用 Jackson/Gson）
56         double score = parseScore(response);
57         String feedback = parseFeedback(response);
58
59         String nextNode = (score >= 80 || count >= 3) ? "exit" :
60         "revise";
61         return Map.of(
62             "review_score", score,
63             "review_feedback", feedback,
64             "iteration_count", count + 1,
65             "next_node", nextNode
66         );
67     }
68 }
69
70 // 修改节点
71 public static class ReviseNode implements NodeAction {
72     @Override
73     public Map<String, Object> apply(OverAllState state) throws
74     Exception {
75         // 将控制流引导回 DraftNode, DraftNode 会从状态中读取 feedback
76         return Map.of("next_node", "draft");
77     }
78 }
79
80 // 输出节点
81 public static class ExitNode implements NodeAction {
```

```

@Override
public Map<String, Object> apply(OverAllState state) throws
Exception {
    String draft = state.value("current_draft").map(v ->
(String) v).orElse("");
    return Map.of("output", draft);
}
}

```

第三步：组装 Graph

```

1 public static CompiledGraph buildWorkflow(ChatModel chatModel) java
2 throws GraphStateException {
3     ChatClient.Builder builder = ChatClient.builder(chatModel);
4
5     var draft = node_async(new DraftNode(builder));
6     var review = node_async(new ReviewNode(builder));
7     var revise = node_async(new ReviseNode());
8     var exit = node_async(new ExitNode());
9
10    StateGraph workflow = new
11    StateGraph(createKeyStrategyFactory())
12        .addNode("draft", draft)
13        .addNode("review", review)
14        .addNode("revise", revise)
15        .addNode("exit", exit);
16
17    // 顺序边
18    workflow.addEdge(START, "draft");
19
20    // 条件边：根据 next_node 字段决定路由
21    workflow.addConditionalEdges("draft",
22        edge_async(state ->
23            (String) state.value("next_node").orElse("review")),
24        Map.of("review", "review"));
25
26    workflow.addConditionalEdges("review",
27        edge_async(state ->
28            (String) state.value("next_node").orElse("exit")),
29        Map.of(
30            "revise", "revise",    // 审核不通过 → 修改
31            "exit", "exit"        // 审核通过或达到上限 → 输出
32        ));

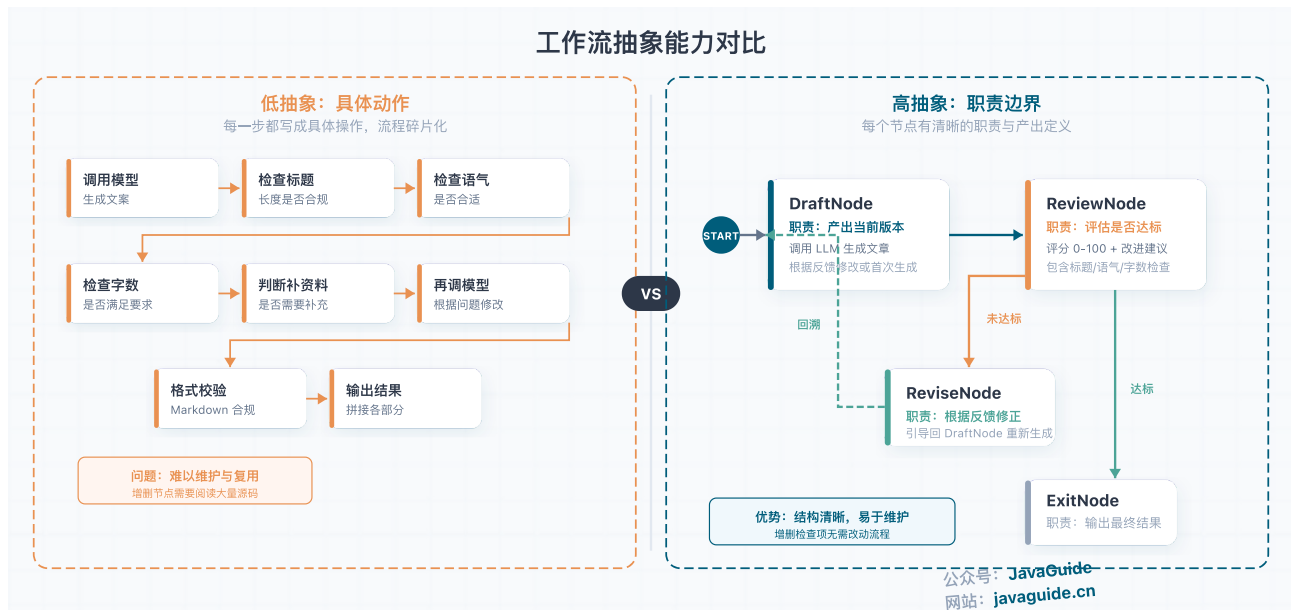
```

```
32
33 // 修改后回到生成节点，形成循环
34 workflow.addConditionalEdges("revise",
35     edge_async(state ->
36         (String) state.value("next_node").orElse("draft")),
37     Map.of("draft", "draft"));
38
39 workflow.addEdge("exit", END);
40
41 return workflow.compile();
}
```

在这个实现中，可以看到：Node 封装执行逻辑，Edge（条件边）控制路由，State（next_node、iteration_count、review_score）驱动决策，Loop 通过 review → revise → draft 的回边实现，安全边界由 iteration_count >= 3 保证。

更完整的示例（包括人机协同、持久化、流式输出）可参考 [Spring AI Alibaba Graph 官方文档](#)。

七、工作流设计的分水岭：抽象能力



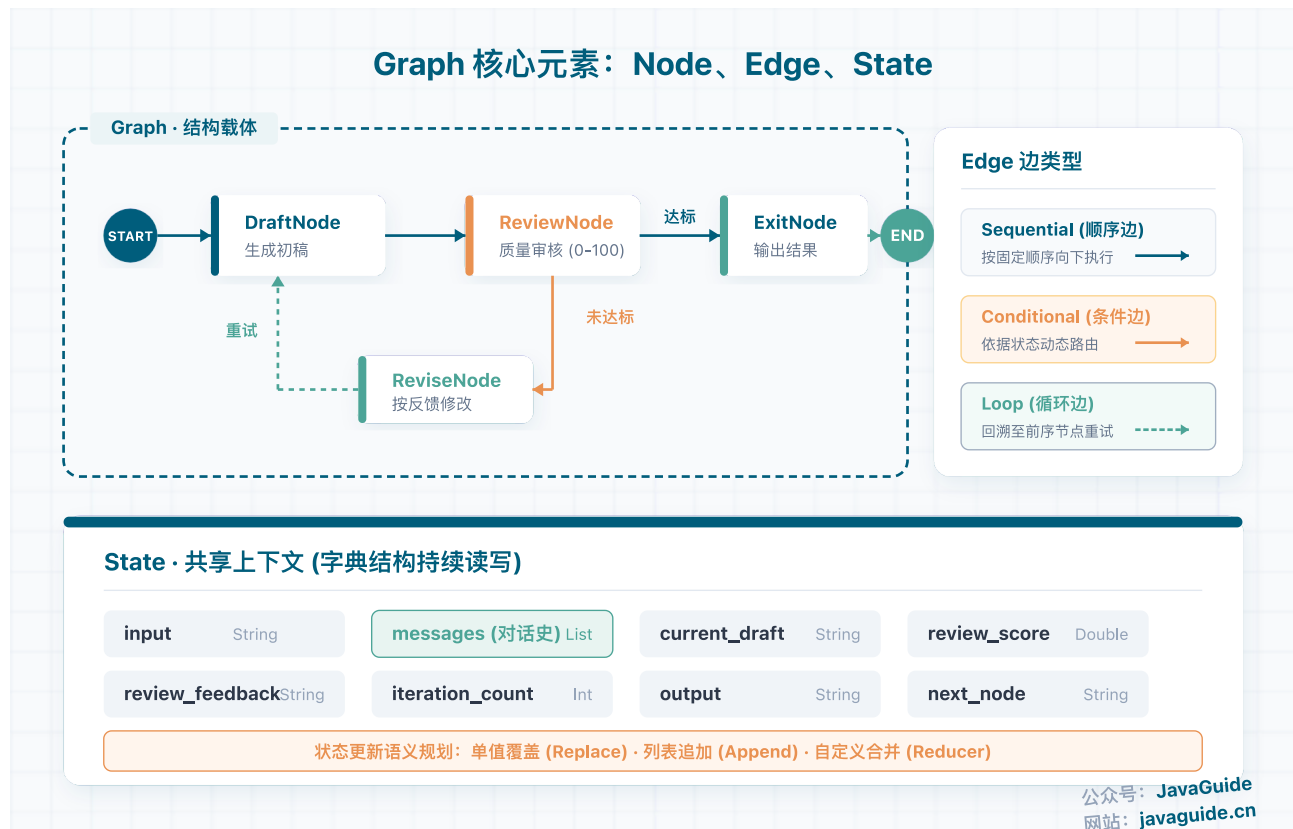
上图可以看到高抽象工作流将四个判断节点抽象成一个判断节点：评估是否达标。如果使用低抽象，那么当我们需要减少/添加新的判断节点时，需要花费时间去阅读源码寻找对应的节点。好的工作流不在于步骤多少，而在于 Node、Edge、State 的抽象是否经得起复用与扩展。

很多初学者设计工作流时，容易把每一步都写成具体动作，例如：调用模型生成文案；检查标题长度；检查语气是否合适；判断是否需要补资料；再调用模型修改。这样做短期可用，但流程会越来越碎，复用性也很差。更成熟的方式是把流程抽象到更稳定的结构层：

1. **Node 抽象职责边界**：在这个节点中产出的结果该是什么样子的，必须出现哪些信息。而不是抽象“这一次调了哪个 API”。
2. **Edge 抽象流转规则**：在什么状态下允许去哪、何时结束。用条件边表达分支与循环，而不是在图外写满 if-else。
3. **State 抽象推进任务时必须持久记住的信息**：工单快照、审核结论、重试次数、错误码等，让路径有据可依。

例如在“生成并审核文章”的场景里，与其设计十几个零散节点来检查文章标题是否符合题意、文章字数是否满足要求，不如先抽象出几个更稳定的职责：

- DraftNode：负责产出当前版本内容。
- ReviewNode：负责评估当前结果是否达标。
- ReviseNode：负责根据反馈修正内容。
- ExitNode：负责在满足条件时输出最终结果。



八、设计工作流时的注意事项

真正把工作流落地时，问题往往不出在“图不会画”，而出在细节没有提前设计好。下面这些是实践里最常见的坑。

1. State 设计的粒度

- 太粗：所有东西都塞进一个大对象里，谁改了哪个字段不好查。
- 太细：字段拆得特别散，每个节点都要拼来拼去，容易出错。
- 建议：按业务含义分几块，例如「用户原始输入一块」「当前生成结果一块」「审核/评分结论一块」「流程控制用的一块（如当前步骤、重试次数）」。

2. 循环终止条件（避免死循环）

不要只写“如果不满意就继续优化”，而要明确：

- 最大轮次是多少？
- 评分阈值是多少？
- 超时或成本超限时怎么办？
- 连续失败后是否要 fallback。

3. 错误处理与 fallback

AI 工作流不是只处理“成功路径”。工具异常、模型超时、格式校验失败、外部接口限流，都应在图上有**明确边**：重试、降级（例如跳过某工具）、转人工、或输出“当前最优 + 错误说明”，而不是只靠外围 try-catch 吞掉。

Spring AI Alibaba 官方文档将错误分为四类，每类对应不同处理策略：

错误类型	示例	处理策略
瞬时错误	网络超时、API 限流	指数退避重试，设置最大重试次数
LLM 可恢复错误	工具调用失败、输出格式异常	将错误存入 State，循环回去让 LLM 根据错误信息调整策略

错误类型	示例	处理策略
用户可修复错误	缺少必要信息、指令不明确	<code>interruptBefore</code> 暂停执行，等待人工输入后恢复
意外错误	未知异常	让异常冒泡，交给开发者调试

这些策略可以直接映射到分布式系统中成熟的弹性模式：

- **指数退避重试**：工具调用超时 → 按 1s、2s、4s 递增间隔重试，设置最大次数（如 5 次），对认证失败等不可恢复错误直接跳过重试。
- **熔断器 (Circuit Breaker)**：连续 N 次 LLM 输出格式校验失败 → 熔断并降级到模板输出或更简单的模型，避免持续浪费 Token。
- **舱壁隔离 (Bulkhead)**：为不同外部 API 设置独立的并发上限，防止某个慢服务耗尽所有工作线程。
- **补偿事务 (Saga)**：多步骤操作中某步失败时，按反序执行已完成步骤的补偿操作（如撤销已创建的工单）。

4. Token 消耗与成本控制

Loop 会自然放大 token 与延迟。设计时要提前思考：

- 哪些节点必须调用大模型，哪些可以用代码替代。
- 是否可以先粗筛，再精修。
- 是否需要在达到“足够好”时就提前结束，而不是追求“理论最优”。

5. 节点间数据传递格式

节点之间传什么、字段名怎么定义、结构化输出采用什么 schema，都应该尽早统一（例如统一用 JSON Schema 或 Pydantic 模型）。否则图一旦复杂，调试成本会急剧上升。

九、总结

用这套视角看问题，工作流就不只是可视化画布上的箭头图，而是一种工程建模能力。常见演进方向包括：

- **Agent 化**：节点从「固定脚本」变成「能自主选工具、拆子目标」的执行单元，但底层仍需要清晰的图与状态边界，否则难以观测与兜底。

- **多智能体协作**：多个角色分工、对话或委托；与 CrewAI、LangGraph 多子图等思路一致，难点往往在**共享 State 的权限与冲突解决**。
- **人机协同**：在关键节点插入人工审核、标注或纠偏，把 HITL (human-in-the-loop) 当作一等公民写进图与状态机。
- **更长上下文与记忆**：工作流与 RAG、会话记忆结合时，要特别注意 State 里哪些该进向量库、哪些只该留在本轮任务上下文，避免成本和隐私失控。
- **Agent 安全**：工作流为 LLM 输出引入了结构和约束，但也带来了新的攻击面。根据 OWASP LLM Top 10，需要重点关注三类威胁：
 - **提示注入的级联影响**：恶意用户输入可能覆盖系统提示，在工作流中逐节点传播放大。防御方式包括输入过滤、系统提示与用户输入严格分隔、对 LLM 输出做安全检测后再传递给下游节点。
 - **工具调用的权限边界**：遵循最小权限原则，每个节点只能访问其任务所需的工具，高风险操作（删除、发送）需通过人机协同节点确认。
 - **输出内容安全过滤**：LLM 输出在进入下游系统（数据库、前端渲染、Shell 命令）前必须经过校验，防止注入攻击、隐私泄露和幻觉传播。

工作流框架会换代，但「图结构 + 状态 + 可控循环」这层抽象会持续存在，所以我们需要深入思考这种思想，摒弃框架思维。

最近更新: 2026/4/13 18:02

贡献者: 6666ccc,Guide